

2.4.5

```
#include <stdio.h>
#include <stdlib.h>

struct polynomial {
    int coeff;
    int exp;
    struct polynomial *next;
};
typedef struct polynomial *poly;

// Function to add a term to the polynomial, combining like terms if needed
poly addTerm(poly head, poly temp) {
    if (head == NULL || temp->exp > head->exp) {
        temp->next = head;
        return temp;
    }

    poly current = head, prev = NULL;

    while (current != NULL && current->exp > temp->exp) {
        prev = current;
        current = current->next;
    }

    // If exponent already exists, combine coefficients
    if (current != NULL && current->exp == temp->exp) {
        current->coeff += temp->coeff;
        free(temp); // No need for an extra node
        return head;
    }

    // Otherwise, insert the new term in the correct position
    if (prev == NULL) { // Insert at head
        temp->next = head;
        return temp;
    } else {
        temp->next = prev->next;
        prev->next = temp;
    }

    return head;
}

// Function to print the polynomial
void print(poly head) {
    poly temp = head;
    while (temp != NULL) {
        printf("%d X^ %d", temp->coeff, temp->exp);
        if (temp->next != NULL) {
            printf(" ---> ");
        }
        temp = temp->next;
    }
    printf(" ---> NULL\n");
}
```

2.4.6

```
struct polynomial {
    int coeff;
    int exp;
    struct polynomial *next;
};
typedef struct polynomial *poly;
```

```
poly addTerm(poly head, poly temp);
poly add(poly head1, poly head2);
void print(poly head);
```

```
poly addTerm(poly head, poly temp) {
    if (head == NULL || temp->exp > head->exp) {
        temp->next = head;
        return temp;
    }

    poly current = head, prev = NULL;
    while (current != NULL && current->exp > temp->exp) {
        prev = current;
        current = current->next;
    }

    if (current != NULL && current->exp == temp->exp) {
        current->coeff += temp->coeff;
        free(temp);
        return head;
    }

    if (prev == NULL) {
        temp->next = head;
        return temp;
    } else {
        temp->next = prev->next;
        prev->next = temp;
    }

    return head;
}
```

```
poly add(poly head1, poly head2) {
    poly result = NULL, temp;

    while (head1 != NULL || head2 != NULL) {
        temp = (poly)malloc(sizeof(struct polynomial));
        temp->next = NULL;

        if (head1 == NULL) {
            temp->coeff = head2->coeff;
            temp->exp = head2->exp;
            head2 = head2->next;
        } else if (head2 == NULL) {
```

```

        temp->coeff = head1->coeff;
        temp->exp = head1->exp;
        head1 = head1->next;
    } else if (head1->exp > head2->exp) {
        temp->coeff = head1->coeff;
        temp->exp = head1->exp;
        head1 = head1->next;
    } else if (head1->exp < head2->exp) {
        temp->coeff = head2->coeff;
        temp->exp = head2->exp;
        head2 = head2->next;
    } else {
        temp->coeff = head1->coeff + head2->coeff;
        temp->exp = head1->exp;
        head1 = head1->next;
        head2 = head2->next;
    }
    result = addTerm(result, temp);
}

return result;
}

void print(poly head) {
    poly temp = head;
    while (temp != NULL) {
        printf("%d X^ %d ---> ", temp->coeff, temp->exp);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

2.4.7

```
struct polynomial {
    int coeff;
    int exp;
    struct polynomial *next;
};
typedef struct polynomial *poly;
```

```
poly addTerm(poly head, poly temp);
poly add(poly head1, poly head2);
poly sub(poly head1, poly head2);
void print(poly head);
```

```
poly addTerm(poly head, poly temp) {
    if (head == NULL || temp->exp > head->exp) {
        temp->next = head;
        return temp;
    }

    poly current = head, prev = NULL;
    while (current != NULL && current->exp > temp->exp) {
        prev = current;
        current = current->next;
    }

    // Merge terms if exponent already exists
    if (current != NULL && current->exp == temp->exp) {
        current->coeff += temp->coeff;
        free(temp);
        return head;
    }

    // Insert the new term in the correct position
    if (prev == NULL) {
        temp->next = head;
        return temp;
    } else {
        temp->next = prev->next;
        prev->next = temp;
    }

    return head;
}
```

```
poly add(poly head1, poly head2) {
    poly result = NULL, temp = NULL;

    while (head1 != NULL || head2 != NULL) {
        temp = (poly)malloc(sizeof(struct polynomial));
        temp->next = NULL;

        if (head1 == NULL) {
            temp->coeff = head2->coeff;
```

```

        temp->exp = head2->exp;
        head2 = head2->next;
    } else if (head2 == NULL) {
        temp->coeff = head1->coeff;
        temp->exp = head1->exp;
        head1 = head1->next;
    } else if (head1->exp > head2->exp) {
        temp->coeff = head1->coeff;
        temp->exp = head1->exp;
        head1 = head1->next;
    } else if (head1->exp < head2->exp) {
        temp->coeff = head2->coeff;
        temp->exp = head2->exp;
        head2 = head2->next;
    } else {
        temp->coeff = head1->coeff + head2->coeff;
        temp->exp = head1->exp;
        head1 = head1->next;
        head2 = head2->next;
    }
    result = addTerm(result, temp);
}

return result;
}

```

```

poly sub(poly head1, poly head2) {
    poly result = NULL, temp = NULL;

    while (head2 != NULL) {
        temp = (poly)malloc(sizeof(struct polynomial));
        temp->coeff = -head2->coeff;
        temp->exp = head2->exp;
        temp->next = NULL;
        result = addTerm(result, temp);
        head2 = head2->next;
    }

    return add(head1, result);
}

```

```

void print(poly head) {
    poly temp = head;
    while (temp != NULL) {
        printf("%d X^ %d ---> ", temp->coeff, temp->exp);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

2.4.8

```
struct polynomial {
    int coeff;
    int exp;
    struct polynomial *next;
};
typedef struct polynomial *poly;

// Function to add a term to the polynomial in sorted order
poly addTerm(poly head, poly temp) {
    if (head == NULL || temp->exp > head->exp) {
        temp->next = head;
        return temp;
    }
    poly current = head;
    while (current->next != NULL && current->next->exp > temp->exp) {
        current = current->next;
    }
    if (current->next != NULL && current->next->exp == temp->exp) {
        current->next->coeff += temp->coeff;
        free(temp);
    } else {
        temp->next = current->next;
        current->next = temp;
    }
    return head;
}

// Function to create a polynomial by taking user input

// Function to multiply two polynomials
poly mul(poly head1, poly head2) {
    poly result = NULL;
    for (poly t1 = head1; t1 != NULL; t1 = t1->next) {
        for (poly t2 = head2; t2 != NULL; t2 = t2->next) {
            poly temp = (poly)malloc(sizeof(struct polynomial));
            temp->coeff = t1->coeff * t2->coeff;
            temp->exp = t1->exp + t2->exp;
            temp->next = NULL;
            result = addTerm(result, temp);
        }
    }
    return result;
}

// Function to print a polynomial
void print(poly head) {
    poly temp = head;
    while (temp != NULL) {
        printf("%d X^ %d ---> ", temp->coeff, temp->exp);
        temp = temp->next;
    }
    printf("NULL\n");
}
```